3-bit x 3-bit Signed Multiplier (mult3x3)

Output Verification & Source Code — Sign-Magnitude Design

1. Objective

This document records the Vivado simulation output of the mult3x3 module, which multiplies two 3-bit signed inputs and produces a 4-bit sign-magnitude output, and lists the complete final source code.

Design approach: per supervisor's instruction, the design uses a sign-magnitude approach rather than plain two's-complement truncation: the sign of each input is separated out, only the magnitudes are multiplied (unsigned), the magnitude result is truncated to fit, and the final sign bit (XOR of the two input signs) is attached at the MSB of the output. Output bit p[3] is the sign (0=positive, 1=negative) and p[2:0] is the truncated magnitude -- this is NOT a two's-complement number.

2. Output Verification

2.1 Simulation Transcript

The transcript below shows 6 sample multiplications. For each, the binary and decimal values of a and b are shown, followed by the raw sign-magnitude output p (binary), its decoded sign and magnitude fields, and the final decimal answer (manually reconstructed from sign+magnitude in the testbench, since p is not two's complement).

```
================================================================================
 mult3x3 demo (sign-magnitude): a x b = answer   (binary and decimal both shown)
================================================================================
a = 011 (3) , b = 010 (2)  ->  p = 0110  [sign=0 mag=110]  =>  answer = 6
a = 101 (-3) , b = 010 (2)  ->  p = 1110  [sign=1 mag=110]  =>  answer = -6
a = 010 (2) , b = 101 (-3)  ->  p = 1110  [sign=1 mag=110]  =>  answer = -6
a = 100 (-4) , b = 100 (-4)  ->  p = 0000  [sign=0 mag=000]  =>  answer = 0
a = 001 (1) , b = 001 (1)  ->  p = 0001  [sign=0 mag=001]  =>  answer = 1
a = 000 (0) , b = 011 (3)  ->  p = 0000  [sign=0 mag=000]  =>  answer = 0
================================================================================
```

Figure 1: Simulation transcript for mult3x3.

2.2 Waveform (Vivado)

The waveform below shows a, b, the raw output p, and the decoded decimal_answer signal across all 6 test cases.

Figure 2: Vivado waveform for mult3x3 (a, b, p, decimal_answer).

Note: decimal_answer's transition in the waveform can appear to lag a/b by one segment. This is a display/update-timing artifact of when the testbench computes decimal_answer within each 10ns test step, not an error in the result -- the console transcript (Figure 1), which prints a, b and the answer together at the same simulation instant, is the authoritative record and confirms each pair is matched correctly.

2.3 Result Summary

a	b	Full product (a x b)	Sign	Truncated magnitude	Final answer
3	2	6	0 (+)	6	6
-3	2	-6	1 (-)	6	-6
2	-3	-6	1 (-)	6	-6
-4	-4	16	0 (+)	0 (overflow, truncated)	0
1	1	1	0 (+)	1	1
0	3	0	0 (+)	0	0

All 6 simulated results match the expected sign-magnitude calculation, including the overflow case (-4 x -4 = 16), where the true product does not fit in the 3-bit magnitude field and is truncated to 0.

3. Source Code

mult3x3.v

Purpose: Multiplies two 3-bit signed inputs using a sign-magnitude approach (separate sign, multiply magnitudes, truncate, reattach sign bit), producing a 4-bit sign-magnitude output.

```
// ================================================================
// Module: mult3x3
// Description: Multiplies two 3-bit signed inputs using a
//              SIGN-MAGNITUDE approach (as requested):
//                1. Separate the sign of each input.
//                2. Multiply only the magnitudes (unsigned).
//                3. Truncate the magnitude product to fit.
//                4. Attach the final sign bit at the MSB.
//              Output p is a 4-bit SIGN-MAGNITUDE value:
//                p[3]   = sign bit (0 = positive, 1 = negative)
//                p[2:0] = truncated magnitude
//              NOTE: this is NOT two's complement -- do not treat
//              p as a normal "signed" number when reading it back.
// ================================================================
module mult3x3 (
    input  wire signed [2:0] a,   // 3-bit signed input  (-4 to +3)
    input  wire signed [2:0] b,   // 3-bit signed input  (-4 to +3)
    output wire        [3:0] p    // 4-bit SIGN-MAGNITUDE output
);

    // ---- Step 1: pull out the sign of each input ----
    wire sign_a = a[2];
    wire sign_b = b[2];

    // ---- Step 2: get the magnitude (absolute value) of each input ----
    // (3-bit two's-complement negate, works correctly even for -4)
    wire [2:0] mag_a = sign_a ? (~a + 3'b001) : a;
    wire [2:0] mag_b = sign_b ? (~b + 3'b001) : b;

    // ---- Step 3: multiply magnitudes (unsigned), then truncate ----
    wire [5:0] full_mag_product = mag_a * mag_b;   // up to 4x4=16, needs up to 5 bits
    wire [2:0] truncated_mag    = full_mag_product[2:0];

    // ---- Step 4: work out the final sign, attach it at the MSB ----
    wire sign_p = sign_a ^ sign_b;

    assign p = {sign_p, truncated_mag};

endmodule
```

tb_mult3x3_demo.v

Purpose: Demo testbench; applies 6 representative input pairs and prints binary + decimal for a, b, and the correctly decoded sign-magnitude answer.

```
`timescale 1ns/1ps
// ============================================================
// Simple demo testbench for mult3x3 (sign-magnitude version)
// Shows both binary and decimal for a, b, and the result.
// p[3]   = sign bit (0=positive, 1=negative)
// p[2:0] = truncated magnitude
// ============================================================
module tb_mult3x3;

    reg  signed [2:0] a, b;
    wire        [3:0] p;

    integer decimal_answer;
```

```
    mult3x3 DUT (
        .a(a),
        .b(b),
        .p(p)
    );

    task show_result;
        begin
            #10;
            // manually decode sign-magnitude -> decimal
            if (p[3] == 1'b1)
                decimal_answer = -p[2:0];
            else
                decimal_answer = p[2:0];

            $display("a = %b (%0d) , b = %b (%0d)  ->  p = %b  [sign=%b mag=%b]  =>  answer = %0d",
                      a, a, b, b, p, p[3], p[2:0], decimal_answer);
        end
    endtask

    initial begin
        $display("================================================================================");
        $display(" mult3x3 demo (sign-magnitude): a x b = answer   (binary and decimal both shown)"
);
        $display("================================================================================");

        a = 3;  b = 2;   show_result;
        a = -3; b = 2;   show_result;
        a = 2;  b = -3;  show_result;
        a = -4; b = -4;  show_result;
        a = 1;  b = 1;   show_result;
        a = 0;  b = 3;   show_result;

        $display("================================================================================");
        $stop;
    end

endmodule
```